

RISC-V CNN Accelerator

Final Project Report

Team members:

113302528 許彧嘉

113501548 林靖恩

Table of Contents

1. Design Completion Summary.....	3
2. Architecture Block Diagram	3
3. Data Memory Map and System Flow.....	4
3.1 Memory Map	4
3.2 Our System Execution Flow	5
4. CPU Design.....	5
4.1 CPU Architecture.....	5
4.2 Supported Instructions	6
4.3 How We Build Instructions In COE file.....	6
4.4 How We Deal With The Hazard Resulting From Pipeline	7
5. CNN Coprocessor Design	8
5.1 CNN FSM Transition Graph	8
5.2 How Our CNN Works In Detail	8
6. CPU-CNN Coordination Mechanism	11
7. Special Design Techniques	12
7.1 High-Throughput Pipelined MAC Engine	12
7.2 Two Read Port Reg File	12
8. Behavioral Simulation Result	13
9. Post-Synthesis Simulation Result.....	13
10. Post-Implementation Simulation Result.....	13
11. STA Summary	13
12. Hardware Utilization	16
12.1 Utilization Screenshot	16
13. AT Product.....	16
14. Hardness / Difficulties and Solutions	17
14.1 CPU Pipeline.....	17
14.2 CNN Memory Access.....	17
14.3 Area too Big.....	18
15. Suggestions / Reflection.....	18

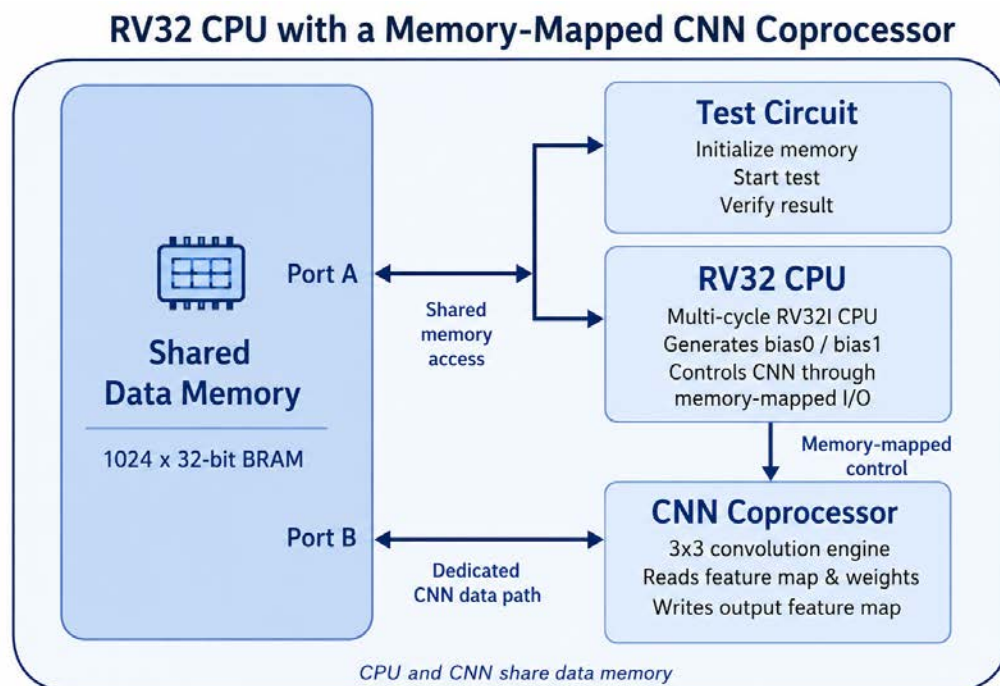
1. Design Completion Summary

Degree of completion:

Requirement	Status
Behavioral RTL simulation	Pass
post-synthesis timing simulation	Pass
post-implementation timing simulation	pass
On board verification	Pass
Pipeline?	Yes
Special Design technique sharing?	Yes

2. Architecture Block Diagram

The system is built around four main blocks: the TA test circuit, a RISC-V CPU, the CNN coprocessor, and connecting through shared Data Memory. The test circuit owns testcase setup and grading. The CPU executes a small instruction image from Instruction_Memory. The CNN reads and writes Data_mem through the second BRAM port.



3. Data Memory Map and System Flow

3.1 Memory Map

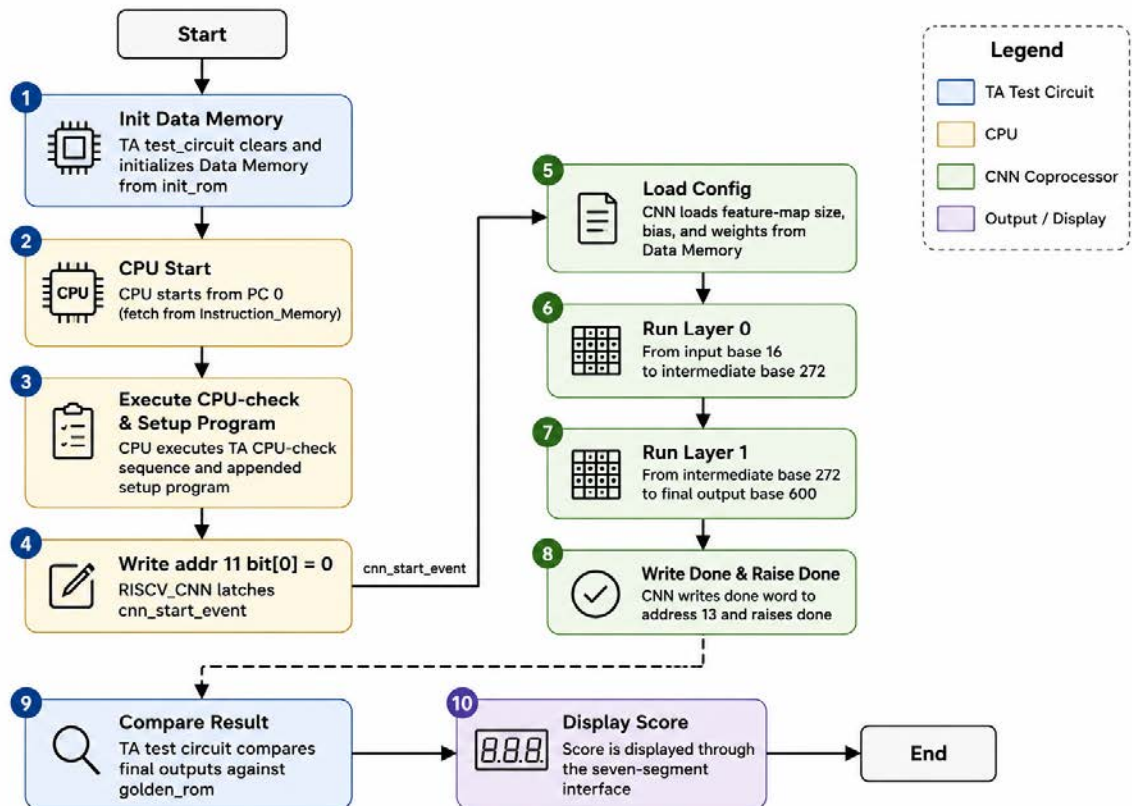
Data Memory Layout (0 ~ 1023)

Address Range	Function / Description	Access / Used by	Notes												
0 ~ 2	L0 Weights 9 signed 8-bit coeffs (3 words)	R CNN	- Depth: 1024 words - Width: 32 bits - Port A: CPU / Test Circuit - Port B: CNN Coprocessor - Predefined regions (except start bit) are read-only - CNN writes results and sets done bit when finished												
3 ~ 5	L1 Weights 9 signed 8-bit coeffs (3 words)	R CNN													
11	Start / Control CPU clears bit0 to start	R/W CPU / Test													
12	Feature-map Size CNN reads [6:1]	R CNN													
13	Output Start & Done [31:11] Output start addr [10:1] Reserved [0] Done bit	R/W CNN / CPU													
14	L0 Bias (written by CPU)	R/W CPU													
15	L1 Bias (written by CPU)	R/W CPU	Control / Config Region												
16 ~ 271	Input Feature-map - Row-major 4 pixels / 32-bit word - Big-endian packing	R CNN	Legend (Access / Function) Weights (Read by CNN) Control / Config (CPU / Test) Data Storage / Processing (CNN R/W) Final Output (Checked by Test) Reserved (Do not use) Input Feature Output Region (272 ~ 599) Final Output Region (600 ~ 959)												
272 ~ 599	Intermediate Output - Output of Layer 0 - Used as input to Layer 1	R/W CNN													
600 ~ 959	Final Output (used by v0.9.7) - Output of Layer 1 - Checked by Test Circuit	R/W CNN / Test													
960 ~ 1023	Reserved (Do not use) - For grading / safety - Do NOT access	— N/A	<table><thead><tr><th colspan="3">Access Summary</th></tr><tr><th>Port</th><th>Connected To</th><th>Access</th></tr></thead><tbody><tr><td>Port A</td><td>CPU / Test Circuit</td><td>32-bit R/W</td></tr><tr><td>Port B</td><td>CNN Coprocessor</td><td>32-bit R/W</td></tr></tbody></table> Both ports access the same memory but with different roles.	Access Summary			Port	Connected To	Access	Port A	CPU / Test Circuit	32-bit R/W	Port B	CNN Coprocessor	32-bit R/W
Access Summary															
Port	Connected To	Access													
Port A	CPU / Test Circuit	32-bit R/W													
Port B	CNN Coprocessor	32-bit R/W													

R = Read Only R/W = Read / Write

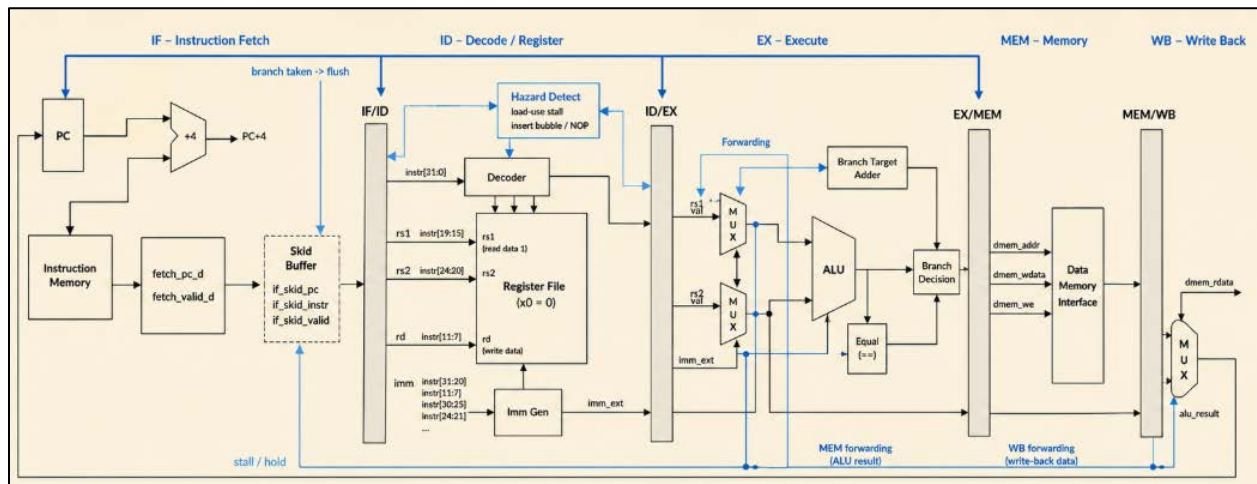
R = Read Only R/W = Read / Write

3.2 Our System Execution Flow



4. CPU Design

4.1 CPU Architecture



We design our CPU architecture according to the one provided in the slides, there are register holding the pervious actions and control signals that travel through each stage.

4.2 Supported Instructions

Instruction class	Opcode / condition	Behavior
ADD	opcode 0110011, funct3=000, funct7=0000000	$rd = rs1 + rs2$
SUB	opcode 0110011, funct3=000, funct7=0100000	$rd = rs1 - rs2$
ADDI	opcode 0010011, funct3=000	$rd = rs1 + imm_i$
LW	opcode 0000011	Read Data Memory then write back
SW	opcode 0100011	Write register value to Data Memory
BEQ	opcode 1100011, funct3=000	Branch if registers are equal
BLT	opcode 1100011, funct3=100	Signed branch if $rs1 < rs2$

4.3 How We Build Instructions in COE file

We build our instruction COE file according to figure 7 and the table above, we transform the assembly code to machine code and then put them underneath the coe file downloaded from eeclass.

1	addi x1, x0, 16	# x1 = x0(0) + 16
2	lw x2, 64(x0)	# x2 = Input feature map,[16]
3	lw x3, 68(x0)	# x3 = Input feature map,[17]
4	lw x4, 72(x0)	# x4 = Input feature map,[18]
5	add x5, x2, x3	# x5 = Input feature map,[16] + Input feature map,[17]
6	add x5, x5, x4	# x5(Bias_0) = Input feature map,[16] + Input feature map,[17] + Input feature map,[18]
7	sw x5, 56(x0)	# 把 Bias_0 放到 Memory Address 14
8	blt x3, x2, T	# if Input feature map,[17] < Input feature map,[16], jump to sub
9	add x3, x2, x1	# x3 = Input feature map,[16] + 16
10	T:	
	sub x4, x2, x3	# x4 = Input feature map,[16] - Input feature map,[17] 或 Input feature map,[16] - (Input feature map,[16] + 16)
11	addi x4, x4, 1	# x4 = bias_1
12	sw x4, 60(x0)	# 把 Bias_1 放到 Memory Address 15

Figure 7 Instruction sequence for bias generation

Hex	Binary	Type	Assembly
04002803,	0000010 00000 00000 010 10000 0000011	I	lw x16, 64(x0)
04402883,	0000010 00100 00000 010 10001 0000011	I	lw x17, 68(x0)
04802903,	0000010 01000 00000 010 10010 0000011	I	lw x18, 72(x0)
011809B3,	0000000 10001 10000 000 10011 0110011	R	add x19, x16, x17
012989B3,	0000000 10010 10011 000 10011 0110011	R	add x19, x19, x18
03302C23,	0000001 10011 00000 010 11000 0100011	S	sw x19, 56(x0)
41180A33,	0100000 10001 10000 000 10100 0110011	R	sub x20, x16, x17
001A0A13,	0000000 00001 10100 000 10100 0010011	I	addi x20, x20, 1
0108C463,	0000000 10000 10001 100 01000 1100011	B	blt x17, x16, +8
FF100A13,	1111111 10001 00000 000 10100 0010011	I	addi x20, x0, -15
03402E23,	0000001 10100 00000 010 11100 0100011	S	sw x20, 60(x0)
02C02A83,	0000001 01100 00000 010 10101 0000011	I	lw x21, 44(x0)
FE0A8EE3,	1111111 00000 10101 000 11101 1100011	B	beq x21, x0, -4
02002623,	0000001 00000 00000 010 01100 0100011	S	sw x0, 44(x0)
00000063;	0000000 00000 00000 000 00000 1100011	B	beq x0, x0, 0

4.4 How We Deal With The Hazard Resulting From Pipeline

Hazard Handling and Forwarding

Our pipelined CPU resolves most RAW data hazards through forwarding. If the required operand is produced by the previous ALU instruction, the result is forwarded directly from the MEM stage to the EX-stage operand input. If the value comes from an earlier instruction, the CPU uses the WB forwarding path. MEM forwarding has higher priority than WB forwarding because it contains the most recent result. The design also includes a WB bypass path in the decode stage to prevent old register values from being read while a write-back operation is occurring.

Load-use Hazard and NOP Insertion

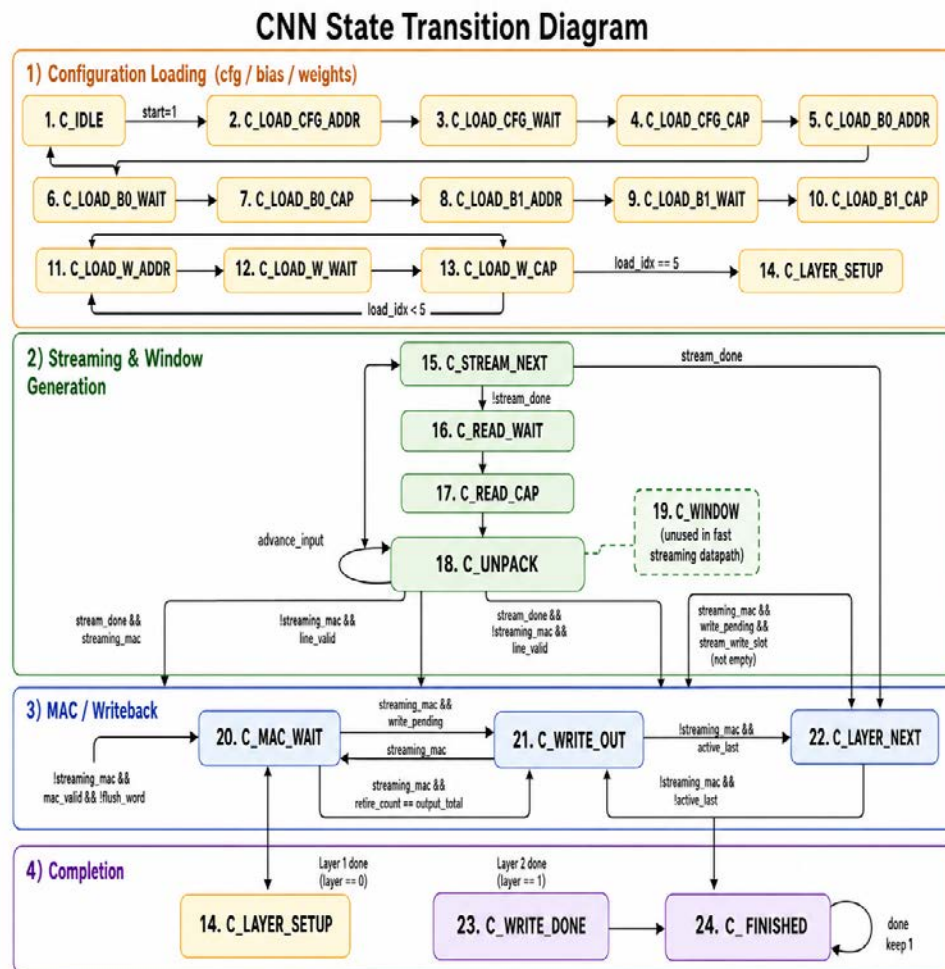
A load-use hazard cannot always be solved by forwarding because the loaded data is not immediately available when the following instruction reaches the EX stage. The CPU detects this condition by checking whether an instruction in the ID stage depends on the destination register of an lw instruction in the EX stage. When such a hazard occurs, the CPU stalls for one cycle and inserts a bubble into the EX stage by clearing the ID/EX valid bit and related control signals. This bubble behaves as a hardware-generated NOP.

Branch Hazard

Branch instructions are resolved in the EX stage. The CPU continues fetching sequential instructions before the branch outcome is known, effectively using a predict-not-taken strategy. If a branch is taken, the PC is redirected to the branch target and the instructions on the incorrect path are flushed by clearing the front-end valid signals and ID/EX control signals.

5. CNN Coprocessor Design

5.1 CNN FSM Transition Graph



5.2 How Our CNN Works In Detail

We will divide our CNN into three main part :

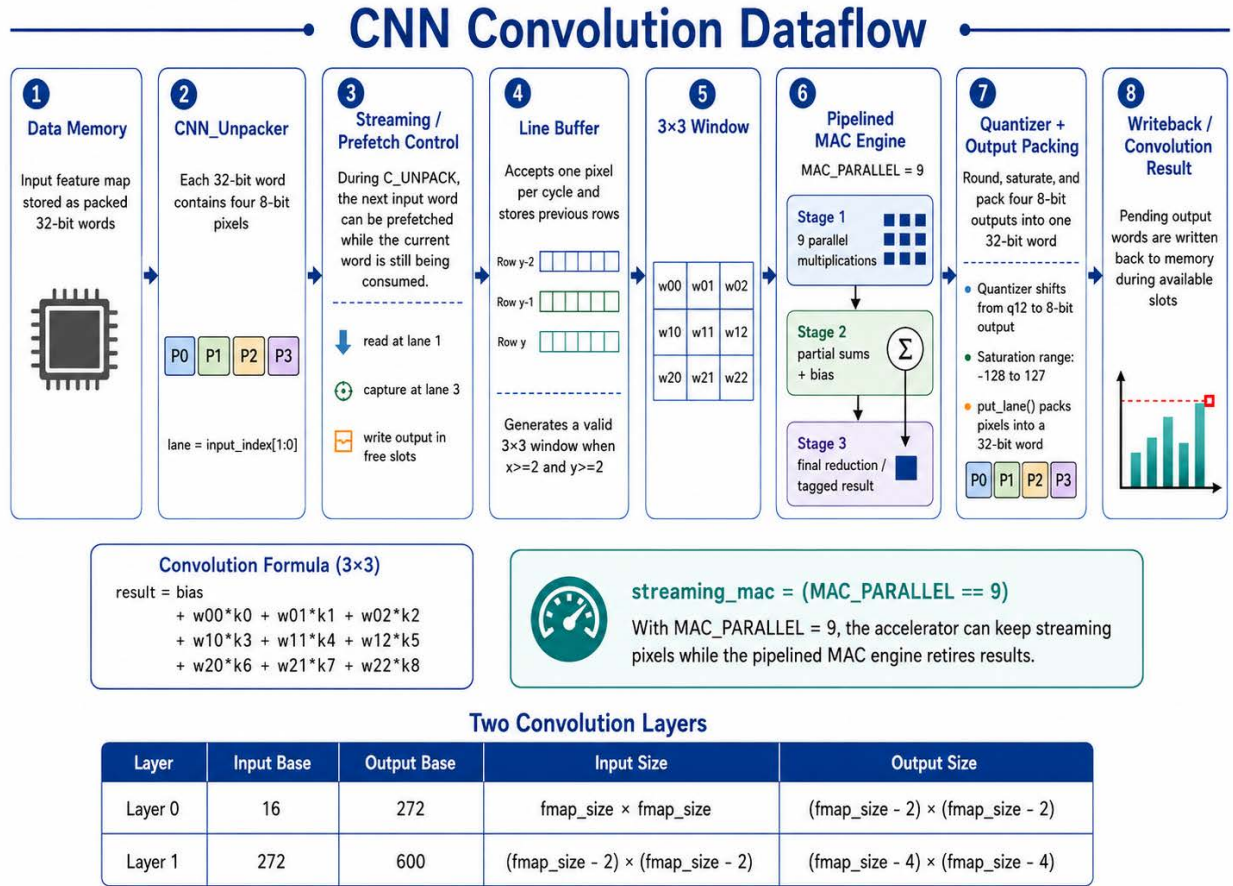
1. Reading configuration
2. Input feature map streaming and MAC operation
3. Output result and write back

1. Read configuration

At the beginning, the CNN module stays in the idle state and waits for the start signal from the CPU. Once the CPU triggers the CNN, the accelerator starts reading required parameters from Data Memory.

The CNN first reads the feature map size from memory address 12. This value is used to determine the input and output dimensions of each convolution layer.

2. Input feature map streaming and MAC operation



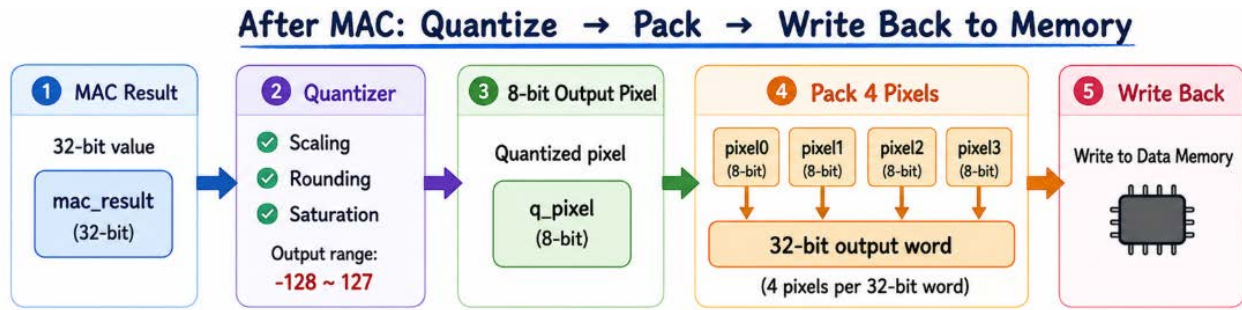
In this stage, the CNN streams the input feature map from Data Memory. The input feature map is stored as packed 32-bit words, where each word contains four signed 8-bit pixels. Therefore, the CNN uses an unpacker to extract one 8-bit pixel at a time according to the current stream lane.

The extracted pixels are sent into the line buffer. The line buffer stores previous rows of pixels and generates a valid 3x3 sliding window once enough pixels have been received. When a valid window is available, the CNN sends the nine window pixels and the corresponding nine kernel weights to the MAC engine.

In our design, we have the MAC engine can perform all nine multiplications in parallel. The MAC engine is also pipelined. This means that after the pipeline is filled, new convolution windows can continue entering the MAC engine while previous results are still being completed. To support this behavior, the CNN uses output tags such as tag_index and tag_last to remember which output pixel each MAC result belongs to .

The streaming design also improves memory access efficiency. Instead of reading a new 32-bit word only after all four pixels are consumed, the CNN prefetches the next input word while it is still unpacking the current word. This allows the pixel stream to run more continuously and reduces unnecessary waiting cycles.

3. Output result



After the MAC operation, the convolution result is still a 32-bit fixed-point value. This value is sent to the quantizer. The quantizer converts the 32-bit MAC result back into a signed 8-bit output pixel. It performs scaling, rounding, and saturation so that the final output remains within the signed 8-bit range from -128 to 127.

After quantization, the CNN packs four signed 8-bit output pixels into one 32-bit word before writing the result back to Data Memory. In MAC_PARALLEL = 9 mode, each result carries tag_index and tag_last metadata so the write-back logic can retire pipelined results in the correct output order and flush the final partial word when needed.

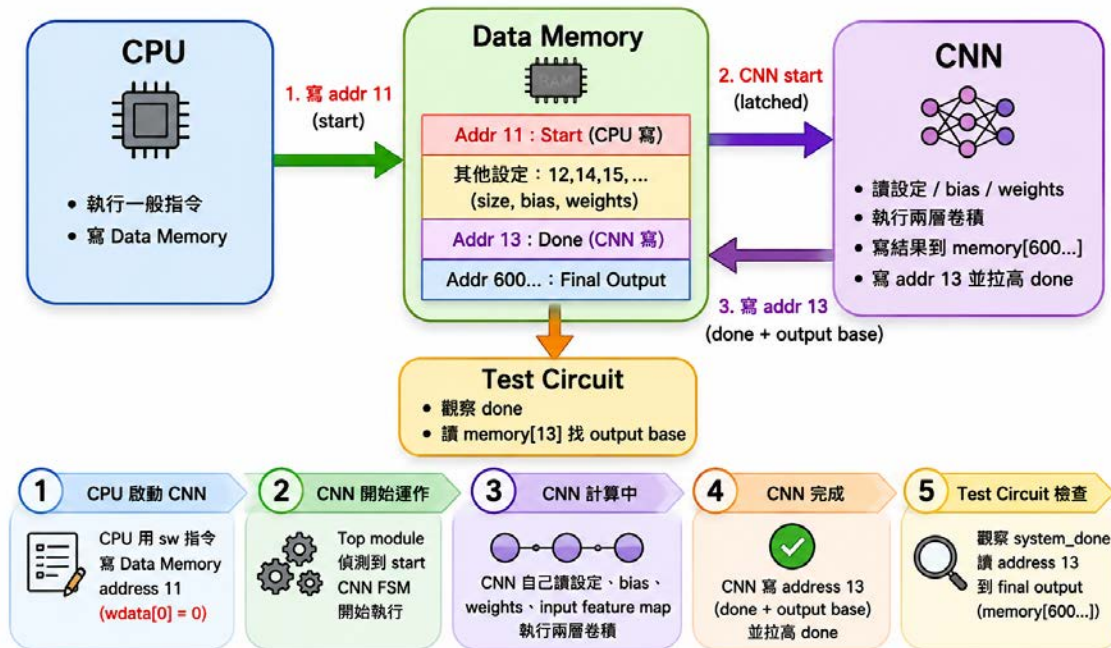
During input streaming, the CNN gives priority to reading the next input word. However, when there is an available memory slot, the CNN writes pending output data back to Data Memory. This helps the design overlap input reading, MAC computation, and output writing.

After the first convolution layer is completed, the CNN switches to the second layer. The intermediate feature map stored at address 272 becomes the input of layer 1. After layer 1 is completed, the final output is stored starting from address 600.

Finally, the CNN writes a done word to Data Memory address [13][0], while bits [10:1] contain the the final output base address, and raises the done signal. The done word contains the final output base address 600 and a done bit, so the test circuit knows where to compare the final feature map.

6. CPU-CNN Coordination Mechanism

CPU – CNN 控制行為 (Memory-mapped)



CPU and CNN communicate through memory-mapped control words rather than direct custom instructions. This keeps the CPU simple and makes the CNN appear like a peripheral on the Data Memory map.

The CPU performs normal store instructions to Data Memory. When the special write to address 11 happens, the top module latches `cnn_start_latched`. The CNN then runs independently and writes completion information to address [13][0]. The TA test circuit observes `system_done` and uses address 13 to locate the output feature map.

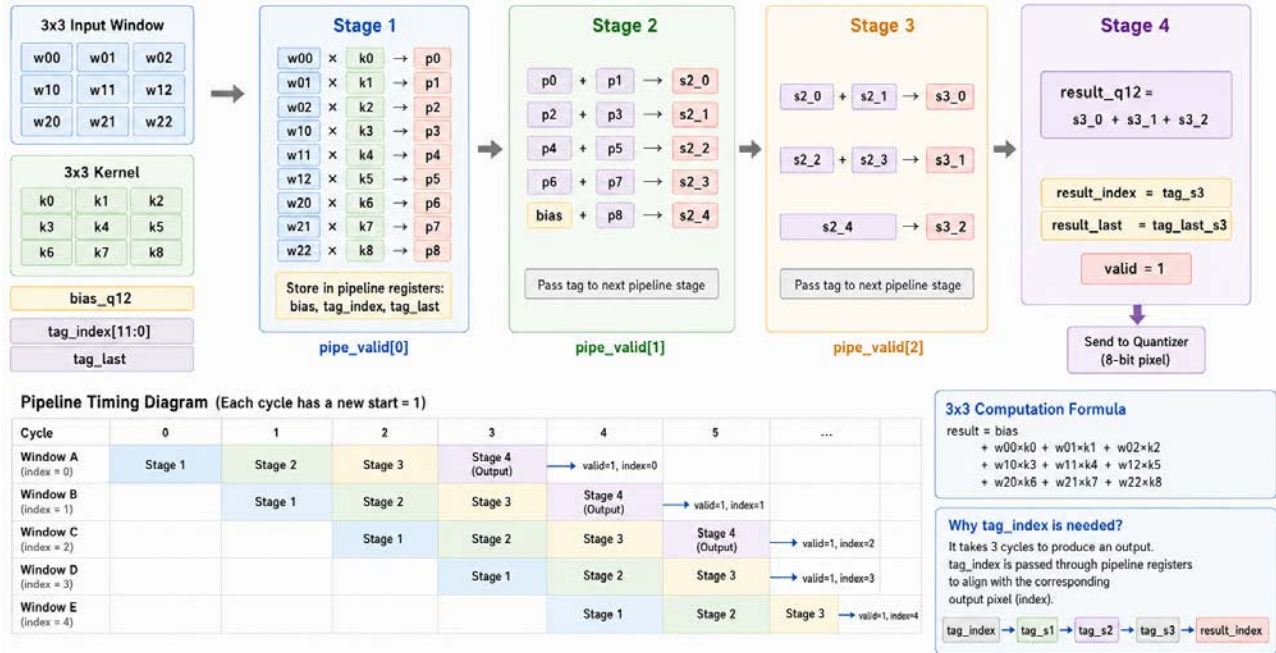
This mechanism has the advantages:

- The CPU does not need a special CNN instruction.
- The CNN can run as a separate FSM with its own Data Memory port.

7. Special Design Techniques

7.1 High-Throughput Pipelined MAC Engine

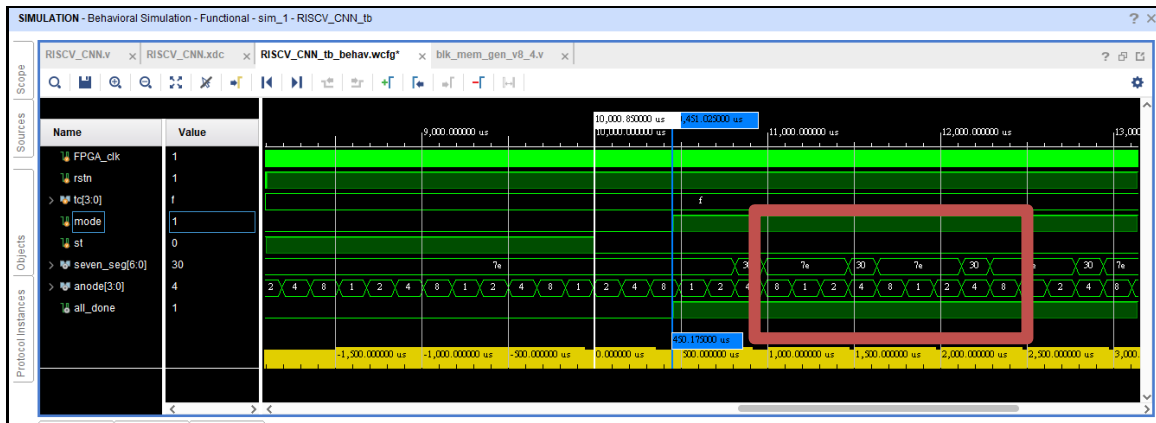
Our CNN accelerator uses a pipelined MAC engine with nine parallel 8-bit multipliers to calculate one 3×3 convolution window. The nine products and the bias are reduced through a multi-stage adder tree, which shortens the combinational path and improves timing performance. After the pipeline is filled, the engine can accept one new convolution window and output one result every clock cycle. Compared with a serial MAC design, this significantly reduces the execution time of the convolution layers.



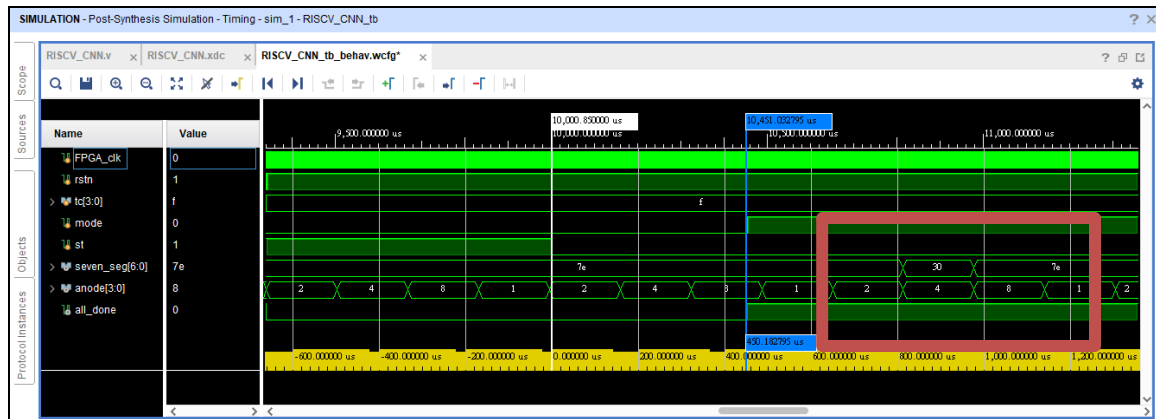
7.2 Two Read Port Reg File

Our CPU uses a two-read-port register file to retrieve rs1 and rs2 simultaneously during the decode stage. The design maintains two synchronized register arrays, allowing both source operands to be accessed within the same clock cycle. During write-back, the new value is written into both arrays to keep their contents consistent. This avoids additional read cycles and helps maintain the throughput of the pipelined CPU. Write-back forwarding is also included so that recently computed values can be used immediately.

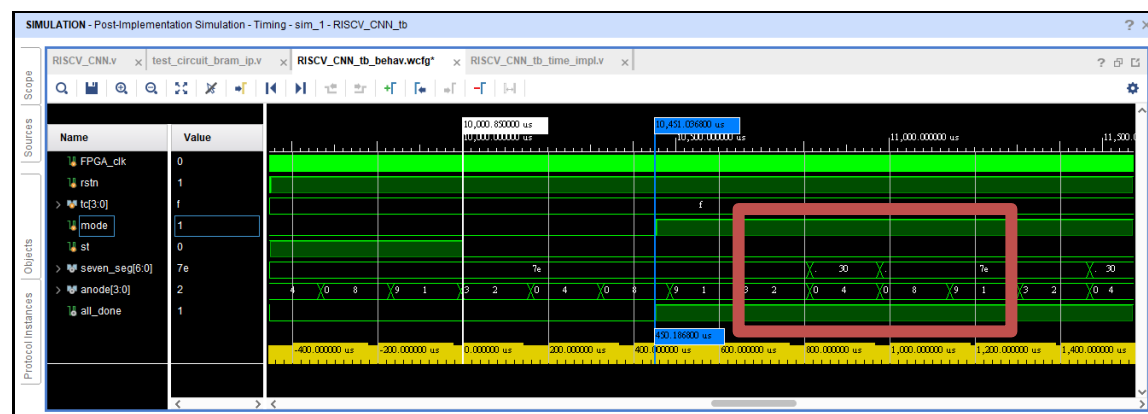
8. Behavioral Simulation Result : PASS



9. Post-Synthesis Simulation Result : PASS



10. Post-Implementation Simulation Result : PASS



$$ProcessingTime = 10451036 - 10000850 = 450186(ns)$$

11. STA Summary

Synthesis STA

Synthesis STA is run at **95.24MHz**, because WNS run under 100 MHz will be negative, according to TA's instruction, it would be ok to lower the clock frequency for synthesis if the design can passed implementation.

SYNTHESIZED DESIGN - synth_1 | xc7a35tcs324-1

Tcl ConsoleMessagesLogReportsDesign RunsTiming x

</

Implementation STA

Implementation STA is run at **100MHz**

IMPLEMENTED DESIGN - xc7a35tcs324-1

Tcl ConsoleMessagesLogReportsDesign RunsDRCMethodologyPowerTiming

Design Timing Summary

General Information

Timer Settings

Design Timing Summary

Clock Summary (1)

Methodology Summary (20)

Check Timing (19)

Intra-Clock Paths

Inter-Clock Paths

Other Path Groups

User Ignored Paths

Unconstrained Paths

Setup

Worst Negative Slack (WNS): 0.169 ns

Total Negative Slack (TNS): 0.000 ns

Number of Failing Endpoints: 0

Total Number of Endpoints: 5342

Hold

Worst Hold Slack (WHS): 0.023 ns

Total Hold Slack (THS): 0.000 ns

Number of Failing Endpoints: 0

Total Number of Endpoints: 5342

Pulse Width

Worst Pulse Width Slack (WPWS): 3.750 ns

Total Pulse Width Negative Slack (TPWS): 0.000 ns

Number of Failing Endpoints: 0

Total Number of Endpoints: 1671

All user specified timing constraints are met.

XDC file screenshot

```
1  # EG01 / xc7a35tcsg324-1 constraints for the final project top module.
2
3  set_property CFGBVS VCC0 [current_design]
4  set_property CONFIG_VOLTAGE 3.3 [current_design]
5
6  set_property PACKAGE_PIN P17 [get_ports FPGA_clk]
7  create_clock -name sys_clk -period 10.000 [get_ports FPGA_clk]
8
9  set_property PACKAGE_PIN P15 [get_ports rstn]
10 set_property PACKAGE_PIN R15 [get_ports st]
11 set_property PACKAGE_PIN P5 [get_ports mode]
12
13 set_property PACKAGE_PIN R1 [get_ports {tc[0]}]
14 set_property PACKAGE_PIN N4 [get_ports {tc[1]}]
15 set_property PACKAGE_PIN M4 [get_ports {tc[2]}]
16 set_property PACKAGE_PIN R2 [get_ports {tc[3]}]
17
18 set_property PACKAGE_PIN B4 [get_ports {seven_seg[6]}]
19 set_property PACKAGE_PIN A4 [get_ports {seven_seg[5]}]
20 set_property PACKAGE_PIN A3 [get_ports {seven_seg[4]}]
21 set_property PACKAGE_PIN B1 [get_ports {seven_seg[3]}]
22 set_property PACKAGE_PIN A1 [get_ports {seven_seg[2]}]
23 set_property PACKAGE_PIN B3 [get_ports {seven_seg[1]}]
24 set_property PACKAGE_PIN B2 [get_ports {seven_seg[0]}]
25
26 set_property PACKAGE_PIN H1 [get_ports {anode[0]}]
27 set_property PACKAGE_PIN C1 [get_ports {anode[1]}]
28 set_property PACKAGE_PIN C2 [get_ports {anode[2]}]
29 set_property PACKAGE_PIN G2 [get_ports {anode[3]}]
30
31 set_property PACKAGE_PIN K2 [get_ports all_done]
32
33 set_property IOSTANDARD LVCMOS33 [get_ports {
34     FPGA_clk rstn st mode all_done
35     tc[0] tc[1] tc[2] tc[3]
36     seven_seg[0] seven_seg[1] seven_seg[2] seven_seg[3]
37     seven_seg[4] seven_seg[5] seven_seg[6]
38     anode[0] anode[1] anode[2] anode[3]
39 }]
```

12. Hardware Utilization

12.1 Utilization Screenshot

Hierarchy

Summary

▼ Slice Logic

▼ Slice LUTs (10%)

▼ LUT as Memory (1%)

LUT as Distributed

LUT as Logic (10%)

▼ Slice Registers (4%)

Register as Flip Flop (4)

Resource	Utilization	Available	Utilization %
LUT	2036	20800	9.79
LUTRAM	60	9600	0.63
FF	1528	41600	3.67
BRAM	14	50	28.00
DSP	1	90	1.11
IO	20	210	9.52

IMPLEMENTED DESIGN - xc7a35tcs9324-1

Tcl ConsoleMessagesLogReportsDesign RunsDRCMethodologyPowerTimingUtilization x

Hierarchy

Summary

▼ Slice Logic

▼ Slice LUTs (10%)

▼ LUT as Memory (1%)

LUT as Distributed RAM

LUT as Logic (10%)

Name	Slice LUTs (20800)	Slice Registers (41600)	Slice (8150)	LUT as Logic (20800)	LUT as Memory (9600)	Block RAM Tile (50)	DSPs (90)	Bonded IOB (210)	BUFGCTRL (32)
▼ N RISC_V_CNN	2036	1528	724	1976	60	14	1	20	1
> u_cnn (CNN)	1154	905	429	1138	16	0	0	0	0
> u_cpu (Simple_CPU)	332	305	118	288	44	1	0	0	0
> u_data_mem (Data_mem)	0	0	0	0	0	1	0	0	0
> u_test_circuit (test_circuit)	544	298	199	544	0	12	1	0	0

The official area formula from the project worklog is:

$$\text{Area} = \text{Slice LUTs} + \text{Slice Registers} + \text{F7 Muxes} + \text{F8 Muxes} + 280 \times \text{DSPs}$$

For our design :

$$\text{Official Area} = 2036 + 1528 + 0 + 0 + 280 \times 1 = 3844$$

13. AT Product (Not included reduce *init_rom loading time*)

$$\text{AT product} = 3844 \times 450186 = \mathbf{1,730,514,984} \text{ (area} \cdot \text{ns)}$$

14. Hardness / Difficulties and Solutions

Hardness : very very very challenging, can't be harder, we are running out of token >_<

14.1 CPU Pipeline

Problem:

The main challenge of the CPU pipeline was that it requires so many control signals to let it function properly, for example, data hazards require different handling depending on the instruction type: ALU results can be forwarded immediately, while load results are not available until the memory response returns. During a load-use stall, the instruction memory may still return a previously requested instruction, which would be lost if the pipeline were simply frozen. Branch instructions introduce an additional issue because the decision is made in the execution stage, after younger instructions have already entered the pipeline.

Solution:

We added delayed fetch registers, valid bits, forwarding paths, a load-use hazard detector, a one-entry skid buffer, and branch flush logic. The delayed fetch_pc_d register aligns each synchronous BRAM response with the correct PC. The forwarding logic prioritizes the newest memory-stage ALU result and falls back to the write-back result when necessary. Load instructions are excluded from memory-stage forwarding because their data is not immediately available; instead, a one-cycle bubble is inserted when a load-use dependency is detected. A skid buffer stores the instruction returned by the BRAM during the stall so that it can be reinserted into the pipeline afterward. When a branch is taken, the PC is redirected and the valid bits of younger instructions are cleared. This allows the CPU to maintain correct execution while reducing unnecessary stalls and preserving pipeline throughput.

14.2 CNN Memory Access

Problem:

Naive convolution rereads overlapping input pixels many times, causing many Data Memory wait cycles.

Solution:

CNN_LineBuffer streams pixels and reuses two previous rows with distributed RAM. After the first word is loaded, the FSM requests the next packed word at lane 1 and captures it at lane 3 while the current word is still being unpacked, and then unpacks four byte lanes locally. This reduces BRAM traffic and allows window generation to proceed in a regular stream.

14.3 Area too Big

Problem:

More parallel MAC hardware using dsp will occupy too many area.

Solution:

We originally forced vivado to use a lot of DSP in the hope of speeding up the process of MAC computation, but this approach was later declared illegal and also led to an enormous area. As a result, we try to avoid using DSP by shortening the number of bits during the computation, which will have less chances for vivado to use dsp and direct to use more LUTRAM (we did not force vivado use LUTRAM). By doing this, we can persist the short processing time while trying to lower the total area.

15. Suggestions / Reflection

林靖恩：

We would hope that the final version of the pdf can be settled a little bit earlier, with less updated version, but overall, we are very grateful toward the TAs' hard work and dedication. This project is very challenging yet interesting for us, but we won't say we look forward to another one.

許彥嘉：

Through this project, I learned how to integrate a CPU, shared memory, and a hardware accelerator into one complete system. The most valuable experience was debugging the timing differences between behavioral simulation and post-implementation simulation. I also learned that hardware design requires not only correct algorithms but also careful memory scheduling, reset handling, and resource control.